

Pi Forge User Guide

Version 0.1.1 | Based on OMP 18.6.2 | October 2026

A practical guide to asking Pi Forge to build, change, understand, and check software.

Pi Forge is an AI coding assistant that works in your project. You describe what you want in ordinary language. It can inspect files, make changes, run commands, test the result, and explain what happened. The AI model supplies the reasoning; Pi Forge supplies the tools, conversation history, settings, and workflow.

Pi Forge keeps the features inherited from OMP and adds its own prompt controls, adaptive task management, context features, and themes. It is a separate application: its executable is `pi-forge`, its application version is 0.1.1, and its updates come from the Pi Forge repository. OMP package names and some service addresses remain for compatibility.

Contents

1. Get started . . . 3

2. Work with Pi Forge . . . 5

3. Choose models and settings . . . 7

4. Understand the coding tools . . . 9

5. Plan and manage larger tasks . . . 11

6. Manage sessions, context, and memory . . . 13

7. Use the web, desktop, and external services . . . 15

8. Customize how the agent works . . . 17

9. Use the Pi Forge additions . . . 19

10. Change appearance and collaborate . . . 22

11. Understand usage and solve problems . . . 24

12. Command reference and glossary . . . 26

How to read this guide

Start with Chapters 1 and 2. Read the other chapters when you need the feature. Chapter 12 is a lookup reference, not a list you need to memorize.

Terminal commands start with `pi-forge` and run in your terminal. **Inside Pi Forge** examples start with `/` and go in the conversation input. Plain sentences in examples are requests you can send to the agent. Text such as `<model-id>` is a placeholder: replace it with your own value, including removing the angle brackets.

Inherited means the feature comes from OMP. **Forge addition** means Pi Forge introduced or extended it. A feature can be installed but disabled, hidden behind a tool menu, or unavailable until its dependency or account is configured. `/tools`, `/settings`, and command help describe what your current session can use.

1. Get started

Install the application

Terminal - Linux x64 with glibc:

```
curl -fsSL https://raw.githubusercontent.com/design-nexus/pi-forge/main/scripts/install.sh | sh
pi-forge --version
```

The installer downloads the published binary, verifies its checksum and version, checks its background workers, and installs `pi-forge` in `~/.local/bin` by default. If your terminal cannot find it, make sure that directory is on your `PATH`, the list of directories your shell searches for programs.

The 0.1.1 binary release supports Linux x64 with glibc. Other platforms use a source build; the fork retains upstream source platform support, but this release does not supply macOS, Windows, ARM64, or musl binaries.

Terminal - build from source:

```
git clone https://github.com/design-nexus/pi-forge.git
cd pi-forge
bun run setup
```

Source builds need Bun 1.4.2 or newer and the Rust toolchain specified in `rust-toolchain.toml`. Bun is the JavaScript runtime and package manager used to build the application; Rust builds its native helper library. Setup creates a source launcher that follows your checkout. You do not need these build tools for the published standalone binary.

Open your project

Start in the folder you want the agent to work on, rather than your home folder.

Terminal:

```
cd /path/to/my-project
pi-forge
```

Alternatively, use `pi-forge --cwd /path/to/my-project`. The startup wizard helps with provider login, model choice, terminal symbols, input layout, and theme. `/setup` opens provider setup again; `pi-forge setup` runs onboarding from a terminal. Additional `setup` subcommands prepare optional runtimes and tools; use `pi-forge setup --help` to see them.

A **provider** is the service running an AI model. Use `/login` for supported account sign-in, or configure the provider's API key using its documented environment variable. An API key is a credential for paid or authenticated service access. Models and features have different account requirements; signing in to one provider does not unlock every integration.

Make your first request

Inside Pi Forge:

```
Explain what this project does. Read the README and main source files.
Do not change anything yet.
```

Then try a small change:

```
Add a short explanation of how to run the project to the README.
Check that the commands match package.json. Show me what changed.
```

You can review the changed file or open `/git` in a Git repository. Ask for tests when code changes need them. A confident final answer is not a substitute for a successful build, test output, or reviewing the result.

Update Pi Forge

Terminal - binary installation:

```
pi-forge update --check
pi-forge update
```

Restart an open session to load the new executable. A source installation is updated by pulling changes in its checkout and rerunning `bun run setup`; `pi-forge update` does not replace the source launcher. Release downloads and update checks use Pi Forge's repository, with no fallback to OMP releases.

Bring settings from OMP

Forge addition. OMP and Pi Forge can stay installed alongside each other. Import is optional and does not remove OMP's data.

Terminal:

```
pi-forge config import-omp --dry-run
pi-forge config import-omp
```

The dry run previews what would be copied. The actual import copies eligible settings and portable login data, keeps existing Forge files and accounts, and leaves OMP originals alone. Add `--sessions --assets` to include saved conversations and custom assets. Use `--from /path/to/.omp` for another OMP configuration root. Some broker-managed credentials are not portable and require another login.

2. Work with Pi Forge

Ask for an outcome, then give constraints

A useful request says what should change, what matters, and how success can be checked. You do not need to explain the implementation if you want the agent to choose it.

```
Build a responsive website for Design Nexus, a web development business.
Use Vue, shadcn-vue, Tailwind, and Bun. Include services, recent work,
and a contact section. Verify the production build and mobile layout.
Tell me whether the contact form actually sends messages or only demos it.
```

For a bug:

```
The Save button does nothing after I change my email address.
Find the cause, fix it, and check that the new address is saved.
Keep the existing page design.
```

For investigation only, say so directly. For a large change, ask for a plan before implementation. Additional instructions sent while the agent is working can steer its active task; `/queue` schedules a message for after it yields.

Attach context

Use `@` to attach a file or image, or select a file through input completion. A screenshot can help explain a visual issue if your selected model supports image input. A text-only model cannot interpret an image merely because the file was attached; image support or an appropriate configured vision path is needed.

Terminal - start with attachments:

```
pi-forge @README.md @screenshot.png "Explain this page and the layout issue."
```

Mention file paths and the exact error when you have them. Pi Forge can read the surrounding project itself, so you usually do not need to paste entire directories into the conversation.

Read the screen

The conversation shows your requests, assistant replies, and tool cards. A tool card is a receipt of an action such as reading a file, running a command, or editing code. Cards may show an intent, progress, output, errors, or a preview. Some long output is shortened on screen or stored as an artifact that the agent can read later.

The status line can show the active model, reasoning effort, Git branch, context, usage, time, background work, and mode. Layout and visibility are configurable. `/context` explains estimated space used in the current conversation; `/jobs` lists background work; `/tools` lists the tools currently shown to the agent.

Interrupt, redirect, and pause

- `Esc` interrupts the current operation. Tell the agent how to continue afterward.
- `/pause` freezes the main agent, workers, and advisor until resumed through the pause controls.
- `/queue` adds a follow-up for the next suitable break.
- `/quit` or `/exit` leaves the application. Saved sessions can be resumed later.

An interruption can arrive while an external command is running or after a file has already changed. Review the current state before assuming everything was rolled back. Pausing, stopping, and switching conversations are different actions.

Useful default shortcuts

Shortcut	Purpose
Enter	Send the current prompt.
Esc	Interrupt the active operation or dismiss a picker.
Ctrl+P / Ctrl+Shift+P	Cycle configured models forward or backward.
Alt+P	Pick a temporary model for this session.
Shift+Tab	Cycle reasoning effort.
Ctrl+T	Toggle the thinking display.
Ctrl+G	Open the prompt in an external editor.
Alt+L	Reset the terminal display after a rendering problem.
Ctrl+D	Exit when the input state permits it.

Run `/hotkeys` for your effective shortcuts: remaps and terminal support can change them. The input also supports history, file completion, configurable multiline behavior, image paste, and optional Vim-style editing. Set the layout and input options under `/settings`.

Use Pi Forge without an interactive screen

Inherited. Print mode handles a task and exits, useful for scripts or a single answer.

Terminal:

```
pi-forge -p "Explain the purpose of src/App.vue without changing files."
```

Piped input can also become the prompt. Print mode can use tools unless you disable them; it is not automatically an analysis-only mode. Use `--no-tools` when no tool access is wanted. Requests needing interactive approval or a user answer may fail without an interactive screen.

3. Choose models and settings

Model choice and reasoning effort

Inherited. `/model` opens model selection; `/switch` temporarily changes the model. The catalog includes hosted APIs, supported coding-plan logins, cloud platforms and gateways, and local engines. Available choices depend on credentials, provider discovery, and enabled models. Use `pi-forge models` to inspect available models instead of relying on a fixed model count.

A **model role** assigns a model to a kind of work. Roles such as default, smol, slow, plan, task, advisor, vision, judge, memory, and speech are used by different parts of the app. Not every role appears as a launch flag. Configure roles with model selection/settings; `--model`, `--smol`, `--slow`, and `--plan` override common roles for a launch.

Reasoning effort is how much deliberate thinking a supported model is asked to do. `/effort low`, `/effort medium`, and `/effort auto` are common choices. Models differ in the levels they support, and higher effort can take longer and produce more billed work. Hiding thinking blocks changes the display, not the model's reasoning cost.

Inside Pi Forge:

```
/effort low
/model
/modelpreset list
```

A model preset saves role assignments and thinking level together. `/fast` requests supported priority service; `/slow` requests supported slower or lower-priority service. These are provider-specific service controls, not universal price guarantees. `/extended-context` requests a larger context window only where the model/provider supports it.

Use LM Studio or another local engine

Pi Forge supports local discovery for engines such as LM Studio, Ollama, and llama.cpp. Start the engine's server, load a model, then inspect the Pi Forge model list. LM Studio defaults to the OpenAI-compatible address `http://127.0.0.1:1234/v1`; `LM_STUDIO_BASE_URL` changes it. Authentication is optional for the default local LM Studio discovery path.

Terminal:

```
pi-forge models find lm-studio
pi-forge --model lm-studio/<model-id> --thinking low
```

Copy the actual model ID from your available list. The local model must support the tool-calling format your server provides to do coding work reliably. A model that answers chat questions can still struggle with tool arguments or long context. Local inference avoids hosted per-token charges but uses your hardware, electricity, memory, and time. Give slower models time to respond before deciding they are stuck.

If your server is not built in, define a custom provider in `~/.pi-forge/agent/models.yml`. It can specify a base URL, credentials, model entries, discovery, and compatibility options. Refer to Model configuration for the exact shape; technical documents may retain OMP path examples, which should be adapted to Pi Forge's configuration root.

Settings and three different kinds of profile

`/settings` is the main settings menu. It exposes common controls; `pi-forge config` also handles settings that do not have a menu editor.

Terminal:

```
pi-forge config list
pi-forge config get adaptive.mode
pi-forge config set adaptive.mode inspect
pi-forge config reset adaptive.mode
```

`get` reads a setting, `set` changes it, and `reset` removes your override. Setting commands normally write global settings; project settings outside dedicated model-role handling are edited in the project file. Use a `--config` overlay to try settings for one launch without changing saved preferences.

Profile type	What it changes
Named application profile: `--profile work`	Separates the app's user-level settings, credentials, sessions, and other profile data.
Model preset: `/modelpreset`	Saves a combination of models and reasoning effort.
Prompt profile: `/prompt setup`	Chooses which instruction modules and capability guidance the agent receives.

Default locations are `~/.pi-forge/agent/config.yml` for global settings and `.pi-forge/config.yml` for project settings. Named application profiles normally use `~/.pi-forge/profiles/<name>/agent/`. XDG layouts and directory overrides can change those paths. `PI_FORGE_CONFIG_DIR`, `PI_FORGE_AGENT_DIR`, and `PI_FORGE_PROFILE` are Forge overrides; explicit legacy overrides remain supported.

Settings combine defaults, global and project choices, launch overlays, and session changes. Some environment variables override those layers, so a saved menu value may not be the effective value. Inspect settings with `config get` when something behaves differently than expected.

Approvals and accounts

Approval mode controls when tool actions ask first. This is separate from asking the agent to plan.

Mode	Normal behavior
`always-ask`	Reads proceed; writes and execution ask first.
`write`	Reads and writes proceed; execution asks first.
`yolo`	Reads, writes, and execution normally proceed automatically. This is the inherited default.

Tool policies and explicit denials can still apply. Approval settings are not a filesystem sandbox: allowed commands and code can use the host access available to the process.

Terminal - choose an approval mode for one session:

```
pi-forge --approval-mode write
```

Supported providers can have multiple accounts and account-selection policies. `/usage` reports limits when the provider exposes them. Privacy, transport, request timeouts, retry/fallback chains, sampling, vision handling, and enabled-provider lists are advanced settings; adjust them for a specific need rather than changing all defaults at once.

4. Understand the coding tools

Files, search, and editing

Inherited. The agent chooses tools from your request; you usually do not type their arguments yourself.

- **Read:** inspect files, folders, documents, URLs, archives, and supported databases. Large source files can be summarized structurally, with detail available when requested.
- **Search:** `glob` finds file names; `grep` finds matching text. `find` searches by a description of behavior and uses a judge model to assess matches, which can incur model usage.
- **Write and edit:** create files or change existing content. The edit tool uses line/content anchors to detect stale reads before applying a patch.
- **Structural search and changes:** `ast_grep` and `ast_edit` work with code structure, useful when a change must match actual expressions rather than similar-looking text. Structural search is off by default; structural edits are enabled but can require discoverable-device access.

```
Find where the app validates email addresses. Explain the current rule,
then change it to allow uppercase letters. Check the affected behavior.
```

Some edits are proposed first and need a resolution step to apply. A preview is not proof that a change is on disk. Optional edit auto-repair can ask another model to fix syntax damage; it is off by default and is separate from Forge's integration repair policy.

Terminal execution and background services

The `bash` tool runs project commands. Pi Forge can shorten verbose output, use an interactive terminal connection, and move eligible long-running commands into background jobs. Ask it to start a development server, run a build, or inspect a command error.

Named supervised services can be reused within a project. `pi-forge ps` lists them, and its subcommands inspect logs or stop, kill, and restart processes. `/jobs` shows asynchronous agent jobs; it is different from the supervised-process list. A live server may need a deliberate stop even after a browser test finishes.

Persistent Python and JavaScript

The `eval` tool runs code in a reusable Python or JavaScript workspace. Variables and helper objects can remain available between cells. This helps with data analysis, repeated browser checks, and workflows that would otherwise require many separate tool calls.

It can call agent tools, define reusable callable tools, run workers, and use configured helpers for documents, archives, databases, Git, judgments, and other integrations. Python requires a working interpreter/runtime; use `pi-forge setup python --check` to diagnose it. A broken or reset kernel may need to recreate variables and browser handles.

Code intelligence, debugger, and binary analysis

LSP is a language-service connection: think of the navigation and error checking found in a code editor. It can find definitions, references, symbols, rename a symbol, and report errors after edits. It is enabled by default, but supported language servers must be installed/configured. `--no-lsp` disables it for a launch.

Debugging can connect to a compatible debug adapter, pause at breakpoints, step through code, and inspect variables. Use `/debug` or ask the agent to investigate a reproducible bug with the debugger. Adapter and project launch configuration determine which programs can be debugged.

IDA Pro integration inspects compiled programs rather than ordinary source. The `ida` tool can open and script analysis databases when a suitable IDA Pro/idalib installation exists. The feature is inert without that installation; it is not a bundled free replacement for IDA.

Check the work and review changes

Ask for the relevant build, tests, or checks, and inspect their results. `/cleanse` can find and repair project diagnostics through workers. `/review` runs a code review of a chosen change set and reports issues by priority. `/annotate code-review` lets you attach your own notes to a diff before the review; `/annotate` also works with other text sources.

The Git interface, available through `/git` or `pi-forge git`, supports inspecting differences, staging, and composing commits. `pi-forge commit` can generate commit messages, split unrelated changes, and handle changelog work according to its options. Inspect its help and proposed scope before using it on a working tree with unrelated changes.

Git conflicts can be inspected and resolved through the same file tools using `conflict://` resources. Ask the agent to explain both sides and choose a resolution; do not assume choosing one side preserves the other side's intent.

Security scanning

Optional security scanning is off by default. Enable `security.enabled` through settings/configuration, then use `/security` to plan, run, inspect, import/export, validate, or compare scans. Cloud scanning requires the supported service and credentials. Results are findings to review, not certification that the project is secure.

See Chapter 12 for the complete tool reference and Tool documentation for detailed formats.

5. Plan and manage larger tasks

Plan mode

Inherited. `/plan` switches to planning before implementation. The agent explores with the permitted tools and proposes steps. Review the plan, request changes, and approve implementation through the plan controls. `/plan-review` reopens the current plan review. You can keep planning until the intended result is clear.

```
Plan a password-reset feature. Explain the screens, data changes,
email delivery, failure cases, and how we will verify it.
```

Plan mode can use a dedicated plan model. The separate `--plan-yolo` launch option automatically approves the proposed plan and moves to implementation; use ordinary plan review when you want to decide yourself.

Todos, goals, and repeated work

Todos track steps and phases. `/todo` shows the list; its edit and mutation commands adjust it. A checked box is an agent-maintained status, not independent evidence that the work passed tests. The details matter when many steps are in progress at once.

Goal mode gives the session a persistent objective to work toward. `/guided-goal` helps you describe one through a conversation; `/goal` manages it, including pause, resume, budget, and drop controls. `--goal` can start a new interactive goal session. A goal budget limits that mode's work; it is not a guarantee of the final outcome or an accurate measure of subscription quota.

Loop mode repeats the next submitted prompt after the agent yields. `/loop` can bound repetition by count or duration, or use a command's exit status as a condition. Press Esc to suspend an active loop and toggle the mode off when done. Goal mode and loop mode solve different problems: one tracks an objective, the other repeats a directive.

Workers and task isolation

A **subagent** is another agent working on a part of your task. The main agent can delegate independent research, implementation, review, or verification and collect results. Workers can use different assigned models. `/agents` configures agent definitions and overrides; `/hub` shows the live Agent Hub; `/jobs` shows background status.

Subagents can be started from the `task` tool or reusable `eval` helpers such as `agent()` and `workpool()`. Work pools share a concurrency limit and can schedule jobs with dependencies. The `wait` tool gathers results and handles peer communication. More workers can cost more and are not always faster, especially on a local server serving them one at a time.

Isolation gives workers separate checkout copies so they do not all edit the same files. It is off by default. Enable `task.isolation.enabled` when you want that workflow; Forge checks integration outcomes before isolated batches are accepted. A **worktree** is another working copy connected to a repository. `/wt` moves the current session into one, including its changes; `pi-forge worktree` manages worktrees separately. Inspect the help before clearing them.

Advisor and model handoff

An **advisor** is an optional second model that watches work and supplies notes. It is off by default. Configure its model role, then use `/advisor configure` or `/advisor on`. It has its own context and usage. It can spot problems, but its advice is not a test result.

Prewalk is an optional one-time model handoff: a starting model explores/plans, and a configured target continues when editing starts. `/prewalk` arms it; `/prewalk restart` returns to the configured default and arms the next handoff. It is off by default. This is useful when you deliberately want separate planning and execution models.

Vibe mode and side work

In **Vibe mode**, the main agent acts as a director. Persistent `fast` and `good` worker sessions do the coding; the director reads and coordinates their results. `/vibe` toggles it. Worker tiers select agent definitions and configured model roles, not a promised response speed. Vibe cannot coexist with active or paused plan/goal modes, and some session-changing actions are blocked while it is active.

`/btw` handles a side question without making it the main task. `/tan` runs tangential work through a background agent. Use them for a related investigation that should not derail the active change.

Special words and automated improvement

These standalone words in an ordinary request add workflow guidance when their required tools are available:

Word	Meaning
`ultrathink`	Ask for especially careful reasoning; automatic effort selects the highest supported level for that turn.
`orchestrate`	Ask for substantial independent parts to be delegated and checked.
`workflowz`	Ask for an explicit multi-worker workflow using persistent execution helpers.
`jevify`	Ask for a fixed rubric and batch judging of a large list, then detailed inspection of flagged items.

They are not activated inside code blocks, paths, or embedded markup. They can increase work and model usage; they do not replace clear instructions.

For advanced users, `/ratchet` helps build an evaluation for an AI workflow and iteratively improve it. The feature is off by default; the command enables its helper for the current session. It runs real evaluation work and can make additional model requests. `/green`, a bundled command, prepares a request to work through CI failures until the branch is green.

6. Manage sessions, context, and memory

Save, resume, and branch

Inherited. A **session** is a saved conversation and its tool history. Ordinary interactive sessions are saved unless persistence is disabled. `/session info` shows identity; `/rename` changes its title; `/pin` keeps it near the top of the recent-session list.

Terminal:

```
pi-forge --continue
pi-forge --resume
```

`--continue` opens the previous session; `--resume` opens a picker, or accepts an ID/path. `/resume` switches from an open session. `--from-claude` and `--from-codex` import supported foreign conversations into Pi Forge. `--no-session` starts an unsaved run and cannot be combined with resume/import operations.

`/branch` returns to an earlier conversation point while retaining the old path as a branch. `/tree` navigates those paths. `/fork` makes a separate session from an earlier point. Conversation branching does not automatically undo files or shell actions. Optional checkpoint/rewind tools operate on context checkpoints, not general repository backups.

`/new` starts a fresh session; `/clear` clears context in the current session; `/delete` deletes the current saved session and starts another. These are distinct operations. `/move` changes the session's working directory. `/add-dir`, `/remove-dir`, and `/dirs` manage additional workspace directories.

What context means

Context is what the model sees for its next answer: instructions, conversation, tool descriptions, and selected results. It has a size limit called a **context window**. Saved history can be much larger than the part currently sent to the model.

`/context` estimates the current breakdown. `/compact` condenses the conversation so work can continue; normal automatic compaction is enabled by default. Its alternate modes depend on your provider and configuration. `/shake` drops selected heavy context, such as images, tool results, or thinking. `/handoff` produces a continuation summary and compacts in place.

Summaries can omit detail. Ask the agent to keep decisions, current state, and verification commands in a file for a long task. The optional archive helper can inspect prior prompts, sessions, recaps, and projects. Internal history resources can recover original journal content where supported.

Memory between sessions

Inherited. Memory is different from keeping the current conversation. It carries selected information into later conversations and is **off by default**.

Memory choice	Plain-language purpose
`off`	Do not use a persistent memory backend.
`local`	Store project summaries and lessons locally.
`mnemopi`	Store structured memories in a local SQLite database.
`hindsight`	Use a configured remote Hindsight memory service.
`sharpshooter`	Keep project decision files and consolidate useful lessons.

Choose `memory.backend` in settings. `/memory view`, `/memory stats`, and `/memory diagnose` inspect available state. Maintenance commands vary by backend; clear/reset operations delete memory data. Hindsight also has mental-model controls under `/memory mm`.

The memory tools can retain, recall, reflect on, or edit memories when the selected backend supports them. The local backend does not provide all structured memory tools. `autolearn.enabled` optionally adds lesson/managed-skill capture guidance after work stops. Both memory generation and learning can require extra model calls. Saved memory can be stale; current files and your latest instructions should take priority.

Recovery during a conversation

`/retry` retries a failed turn. `/fresh` resets provider stream state without clearing the local conversation, useful for stale cached state or a wedged stream. Retry/fallback settings can select another configured model when errors persist; stream loop detection can interrupt repetitive output and repeated tool calls.

`/restart` relaunches with the same flags and resumes the session. Restarting does not necessarily preserve an in-memory Python/JavaScript kernel or restart every external process. Ask the agent to inspect what is still running.

Forge's notes-backed context and relevance features are explained in Chapter 9. They are not the same as long-term memory or ordinary conversation summaries.

7. Use the web, desktop, and external services

Read pages, search, and drive a browser

Inherited. Use the normal `read` tool for static pages, documents, and supported URLs. Web search can use configured providers to retrieve live results and citations. Search availability depends on the provider and its credentials; inspect settings or `pi-forge search --help`. Specialized handlers can process sources such as research pages, code hosting, documentation, and security databases.

The **browser helper** runs inside `eval`. It is enabled by default when eval is available, but needs a supported browser/backend. It can open a headless browser (without a visible window), interact with pages, inspect console output, take screenshots, fill forms, and test different screen sizes. `/browser visible` requests visible mode; `/browser headless` requests hidden mode.

```
Open the local development site. Check the contact form, mobile menu,
and whether the page scrolls sideways at phone and tablet widths.
Capture screenshots and report actual failures.
```

Browser support includes managed Chromium, attached Chromium/Electron through CDP, a relay into an existing Chrome session, and supported terminal-integrated backends. **CDP** is the browser's remote-control connection. `pi-forge browser-relay` prepares the relay; follow its help for the required browser-side setup. Browser access can use real logged-in accounts, so inspect the exact target before authorizing an action that changes outside data.

Control desktop applications

The optional **computer helper** works with the real desktop: windows, screenshots, native keyboard/mouse input, clipboard, and accessibility information. It is off by default; `/computer on` enables it for the session. Capabilities depend on the operating system and desktop backend. This is separate from the browser's webpage interface.

Ask for a concrete task, such as reading an application's visible settings or entering approved text. Use inspection first when you need to confirm the target. Computer automation is not guaranteed to work with every application, permission setup, or window system.

GitHub, remote files, and secret handling

Optional `github.enabled` exposes GitHub operations; authentication and the relevant GitHub CLI/service access are needed. Pi Forge can inspect repositories, pull requests, issues, diffs, and Actions runs. Some resources can be read like files, for example `pr://` and `issue://` paths. Publishing a change or posting a comment is different from reading one.

SSH lets the agent access configured remote hosts through `ssh://` paths. `/ssh` and `pi-forge ssh` manage hosts. SSH connectivity, keys, and host permissions remain necessary; a configured host entry is not a successful connection by itself.

Configured secrets can be kept in `secrets.yml`; optional `secrets.enabled` helps obfuscate configured values before model requests. Sharing and streaming have their own redaction paths. Redaction is not proof that every sensitive value has been found. `/dump` can write request details for troubleshooting, and `pi-forge token` prints credentials: treat those outputs as sensitive.

An optional `vault://` integration can access Obsidian content through its CLI when enabled and installed. General URL/file tools can also inspect archives, SQLite data, notebooks, and supported document formats without requiring a separate conversational tool for each format.

Images, speech, and voice

Image generation/editing is optional and off by default. Enable `generate_image.enabled` and configure a supported image provider/model; the feature can create artwork or change an input image. `pi-forge images` diagnoses image-publication backends. Publishing image URLs is a separate, disabled-by-default option for transports that fetch images by URL; it can send image data to an external host.

Speech features are separate controls:

- **Speak replies:** `speech.enabled` reads assistant output aloud through speakers.
- **Dictation:** `stt.enabled` uses microphone speech as input, with a configured local or supported cloud engine.
- **Create an audio file:** `speechgen.enabled` enables the `tts` tool, using the selected local/cloud speech backend.
- **Local speech command:** `pi-forge say` speaks supplied text through the local engine.
- **Live voice:** `/live` starts supported Codex-backed real-time voice, with its own service/account requirements.

Speech output and dictation are off by default. Local voice models may need downloads; `pi-forge tiny-models` and `pi-forge setup --help` show relevant preparation options. A text model, image model, dictation engine, and speech engine are different components.

8. Customize how the agent works

Project instructions and reusable prompts

Inherited. An `AGENTS.md` file can explain your project's commands, conventions, and expectations. Pi Forge also discovers supported instruction formats from other coding tools. Put durable project background in instructions rather than repeating it in every request.

A simple instruction file might say:

```
Use Bun for package commands.
Run the production build after changing application code.
Keep the existing visual design unless the task asks for a redesign.
Do not commit or publish changes unless asked.
```

Native Forge configuration lives under `.pi-forge` in the project and `~/.pi-forge/agent` for the user. Explicit OMP/project compatibility remains supported. Discovery has precedence and scope rules; an instruction file in one directory does not necessarily apply to every nested package. Existing technical references still containing `.omp` examples are inherited documentation, not an instruction to use OMP's app data.

Prompt templates and custom slash commands turn repeated requests into shortcuts. Templates can insert arguments. `/annotate` lets you prepare more precise feedback on a reply, message, file, quoted text, or code review. `/skillful` controls whether available skills are listed in the current session's prompt.

Skills, plugins, and marketplaces

A **skill** is a reusable instruction playbook, often stored in `SKILL.md`. A **plugin** packages capabilities such as commands, skills, hooks, tools, agents, or MCP servers. A **marketplace** is a source from which plugins can be discovered and installed.

- `/skills` searches, installs, and updates supported registry skills.
- `pi-forge skill` also manages and publishes registry skills.
- `/plugins` lists/enables/disables installed plugins.
- `/marketplace` manages marketplace sources and installations.
- `pi-forge plugin` installs, links, removes, and manages packages.
- `/reload-plugins` reloads supported plugin contributions without ending the conversation.

The current skill registry is `skills.omp.sh`. Pi Forge retains this inherited service; it is not Pi Forge's update server. Installed code extensions can execute with application access, so install packages whose code and source you intend to trust.

MCP connections

MCP, or Model Context Protocol, is a standard connection between an AI application and an external tool server. It is like plugging an extra toolbox into Pi Forge. A server might expose files, a database, or another service.

Use `/mcp add` to configure one, `/mcp list` to inspect connections, and `/mcp test` to check it. The menu also supports enabling/disabling, reconnecting, OAuth authentication, resources, and prompt listings. Smithery discovery/login commands are available where that service is usable.

Native configuration can live in `.pi-forge/mcp.json` or `~/.pi-forge/agent/mcp.json`. Supported foreign-tool configuration files can also be discovered. A server may need a local executable, network endpoint, credentials, or

approval. MCP tools have their own names and account requirements; merely installing Pi Forge does not activate every possible server.

Rules, hooks, and extensions

Rules give focused guidance for particular files or situations. **TTSR**, Time-Traveling Stream Rules, watches generated output and can interrupt a recurring behavior before the response completes. TTSR is enabled by default, but useful rules must exist. `/omfg` helps turn a complaint into a rule; `pi-forge ttsr` inspects and tests them.

Hooks react to events such as tool calls or completed turns. **Extensions** can add tools, commands, providers, interface elements, or custom logic. `/extensions` shows the Extension Control Center; `/agents` manages task-agent definitions and overrides. Developers can load an explicit extension with `--extension` or restrict loading through `--trusted-extension`.

Use the existing authoring guides for exact formats. Changing a setting or adding a skill is simpler than writing code; write an extension when you need behavior that instructions and settings cannot supply.

9. Use the Pi Forge additions

Prompt profiles and modules

A **system prompt** is the instruction package Pi Forge sends before your conversation. A **module** is one section of that package. Forge lets you choose how optional sections and capability guidance are loaded while retaining required instructions.

Prompt profile	Intended behavior
`full`	Default. Keeps comprehensive inherited guidance.
`minimal`	Removes optional workflow/cleanup sections and loads selected guidance automatically.
`coding`	Keeps coding/testing guidance with delegation and selected capabilities loaded automatically.
`agentic`	Keeps delegation/subagent guidance readily available for coordinated work.
`custom`	Uses your module/capability policy choices; it does not start as a blank unrestricted prompt.

Inside Pi Forge:

```
/prompt setup
/prompt stats
/prompt compare
/prompt inspect --redact
```

Setup configures the profile. Stats shows loaded modules and estimated prompt usage. Compare evaluates profiles against this session's context/tools; it is not a coding-performance benchmark. Inspect shows the assembled instructions; use redaction before sharing them. Required modules remain present even if an override requests otherwise.

Module and capability policies are `always`, `automatic`, or `disabled`. These control instruction/presentation behavior, not a promise that every eligible tool is installed or authorized. Profile names describe their design; none guarantees better coding or lower total cost on every task.

Capability routing

Forge can selectively present eligible tool guidance instead of loading all of it up front. Think of keeping common tools on the desk and bringing out specialized tools when needed. Routing is affected by the profile, tool availability, model constraints, and explicit requests.

Inside Pi Forge:

```
/prompt route browser
/prompt unroute browser
```

Explicit route choices include `lsp`, `subagents`, `debugger`, `github`, `images`, `browser`, and a named MCP tool. A route command reports activation or why it cannot activate. It does not bypass a disabled feature, missing dependency, or approval policy. `/tools` helps distinguish actual tool availability from prompt guidance.

Adaptive Governor

The **Governor** estimates how much coordination and checking a task needs. It uses available evidence about scope, dependencies, risk, runtime pressure, and failures to choose a task band and policy. It can revisit that decision during work instead of using the first estimate forever.

adaptive.mode	What happens
off	Default. Do not activate the adaptive policy.
inspect	Record decisions for inspection without applying the automatic policy.
auto	Apply eligible adaptive decisions during the task.

Terminal - turn on inspection first:

```
pi-forge config set adaptive.mode inspect
```

Terminal - enable automatic policy:

```
pi-forge config set adaptive.mode auto
```

Inside Pi Forge:

```
/prompt governor
```

This shows recorded evidence, task band, effort/coordination policy, and checks. No recorded decision means the session has not produced one yet. A worker count of zero can be intentional: many tasks have no useful independent pieces. Explicit task choices and configured constraints can restrict the policy; available model roles determine what it can actually select.

Context retention, relevance, and experimental notes

Forge adds policies for retaining useful task context and selecting bounded relevant information. Relevance uses available text and dependency relationships; Forge's added relevance selection is not a semantic-embedding engine. Other inherited search tools, such as `find`, are separate features.

The experimental **notes-backed context windows** option, `compaction.experimentalContextManagement`, is off by default. Enable it under `/settings` in Context/Compaction if you want to try it. It exposes `context_notes` to keep a small persistent working notebook and `new_context` to request a fresh working context while leaving raw branch history searchable.

The notebook should record decisions, completed work, current failures, and next checks. It is not infinite memory: notes have a size limit, and recovery depends on the available notes, history, read, and grep tools. A new-context request is an acknowledgement, not proof that the rollover has already completed. Ordinary compaction remains the fallback for unsupported tool configurations.

Orchestration and integration checks

Forge extends coordinated tasks with shared concurrency accounting, dependency ordering, queued jobs, and explicit lifecycle outcomes. Workers can wait for prerequisites rather than all starting immediately. Isolation keeps their edits apart; integration combines accepted changes back into the parent workspace.

An **integration gate** checks the batch's reported result and structured completion state before accepting it. Failed or incomplete integration can trigger eligible repair or review rather than silently being presented as finished. This is particularly useful for several workers changing related parts of a project. It does not make conflicting edits impossible.

Adaptive verification, repair, and review

Verification levels **V0-V4** are bands for increasing checking effort. The Governor carries bounds and risk/failure floors into worker instructions. Lower-risk changes can use focused checking, while larger or failing changes can call for more thorough checks and review.

The host validates structured reports and integration outcomes. It does **not** independently run every command a worker says it ran. Review the recorded verification evidence and actual project checks when correctness matters.

Bounded repair allows another attempt after eligible failures, with safeguards for attempt count, time, tokens, estimated cost, and lack of progress. `task.repairBudget` configures those limits. They apply to that repair workflow, not to every action in the application. Conditional independent review follows eligible work; it is not automatically invoked for every tiny edit.

Observe actual results

Forge persists Governor decisions, usage, worker outcomes, and integration evidence. `/prompt governor`, `/jobs`, `/trace`, and the statistics tools expose parts of that record. Offline benchmark summaries and the real-work observation workflow can inspect saved sessions without calling another model.

Measurements so far are mixed. More capable coordination, observable decisions, and controlled integration are benefits even when prompt-token savings are small. They do not establish a consistent speed or cost advantage over OMP. See Evaluation results and Real-work observation.

10. Change appearance and collaborate

Themes and terminal appearance

Inherited, with Forge themes/integration. `/settings` controls theme, dark/light appearance, symbols, fonts/glyph compatibility, input shape, message display, tool previews, status fields, thinking visibility, and image sizing. Nerd Font symbols look different from Unicode/ASCII-safe choices; use the onboarding glyph scene or settings if you see boxes or missing characters.

Forge offers **Catppuccin Mocha** for dark terminals and **Catppuccin Latte** for light terminals, alongside inherited themes. Their configuration names are `dark-catppuccin` and `light-catppuccin`.

Theme Source - Follow Omarchy reads an installed Omarchy desktop's active palette and follows changes. It does not change desktop settings. Without a valid palette it falls back to the configured terminal theme; temporary bad changes can retain the last valid palette. The default theme source remains Terminal themes.

Supported terminals can render richer native surfaces and inline images. Others use the row-based terminal interface. The app still presents the same workflow, but exact image previews, key combinations, and native layout depend on terminal capabilities. The Forge logo preserves the pi symbol and places smaller gradient FORGE lettering beneath its upper-right edge.

Notifications, voice input, and convenience settings

Notifications can signal completed work or waiting questions. Optional idle recaps summarize where the task stands when you return and may make model calls. Power/sleep controls can keep work from being interrupted by system idle behavior. Input prediction, spelling options, external editing, paste handling, and Vim-style input are configurable conveniences; they do not change your project's source conventions.

Read-summary, output-limit, and preview options control how much detail reaches the agent or appears on screen. Retry/fallback, automatic backgrounding, and timeout settings affect how work proceeds. `/settings` groups these by purpose; not every advanced setting has a menu editor.

Export and encrypted sharing

`/export` writes an HTML copy of a session. `/copy` picks text/code, `/open` opens the last link, and `/dump` copies a transcript and can write request diagnostics. `/dump all` includes multiple agent dumps. These operations are useful for troubleshooting or handoff; inspect the content before sending it elsewhere.

`/share` or `pi-forge share` creates an encrypted sharing link through the configured sharing backend. Sharing is a snapshot, not a live collaborative session. Recipients with the appropriate link/key can read its content; encryption does not make the transcript harmless to distribute.

Live collaboration

`/collab` shares the current session through a relay. `/collab view` makes a view-only link; control-capable links let a guest drive the agent. `/join` or `pi-forge join` joins; `/leave` exits as a guest; `/collab stop` stops hosting. `pi-forge collab` lists local host metadata and its `link` subcommand retrieves the selected link.

Collaboration is different from running subagents: another person connects to the session. Availability depends on relay/service configuration. Give recipients the appropriate link type for the access you intend.

Stream, record, and replay

`pi-forge stream` broadcasts rendered terminal screens and chat to the inherited live service. It needs the supported service account, currently Stencil access. Viewers cannot type into the agent through the screen stream. Start the stream before starting sessions you want attached in the same directory.

`/record` records the current screen locally without requiring a streaming account; run it again to stop. `pi-forge play` replays a recording. `pi-forge clip` uploads one as a public clip. A recording is a screen replay, not the same as a saved resumable agent session.

These inherited integrations may show OMP service addresses and `.ompcast` recording files. They remain compatible by design. Screen redaction covers known secrets and patterns, not every possible private value; streaming chat has separate handling. Check what will be shared before broadcasting or uploading.

11. Understand usage and solve problems

Tokens, costs, and time

A **token** is a small piece of text processed by a model. Input tokens describe material it receives; output tokens describe what it generates. A model request can include prior conversation again, so session totals can greatly exceed the size of the saved text. Cache-read/write counts describe provider-reported reuse and may not be available for every provider.

The displayed monetary cost is normally an estimate from reported usage and configured model prices. It is not necessarily your provider invoice or subscription allowance. Reasoning, auxiliary calls, workers, advisors, title/recap generation, memory, judging, image/audio services, and checks may have separate usage. Local models can report $0 while still taking substantial time and resources.

`/usage` and `pi-forge usage` inspect provider-reported usage/limits where supported. `/stats` or `pi-forge stats` opens local usage statistics; `/trace` opens this session's trace. `/context` estimates current context, which is different from total tokens processed over the whole task. `pi-forge toks` counts text with embedded offline tokenizers; it does not reconstruct an entire live provider request.

For a fair comparison, use the same task, starting files, model, effort, enabled features, and acceptance checks. Record retries, user interventions, workers, and idle breaks. A single successful task is useful evidence, not proof that one application is always better.

Common problems

Problem	First things to try
`pi-forge` is not found	Check installation and `~/.local/bin` on PATH.
The screen still shows an older version	Exit and restart; run `pi-forge --version` and check which executable your terminal resolves.
A model is missing	Check `/login`, enabled providers, and `pi-forge models`; inspect provider help/configuration.
LM Studio cannot connect	Confirm its server is running, a model is loaded, and the base URL/port matches.
The local model seems slow	Allow time for prompt processing; inspect progress/logs before interrupting. Large context and multiple workers can slow it further.
A feature is missing	Check `/settings`, `/tools`, required dependencies/account, and discoverable tools under `xd://`.
Python eval is unavailable	Run `pi-forge setup python --check`; verify the interpreter setting. JavaScript may remain available.
Browser code fails	Ask the agent to read its browser helper documentation; do not assume another browser library's method names work.
The agent repeats itself	Interrupt, give a concrete next step, inspect `/jobs` and `/prompt governor`, and consider `/fresh` for stream-state trouble.
Context is too large	Inspect `/context`; try `/compact`, `/shake`, or a saved handoff before continuing.
A development server stops	Inspect `pi-forge ps` and its logs; a terminated process is not a failed production build.
A build looks successful but output contains errors	Check the actual build exit status; commands piped through `tail` or `grep` can hide it.
Todo state looks wrong	Review `/todo` and actual files/checks; completed tasks are not independent verification.
Missing symbols or broken layout	Use ASCII/Unicode-safe glyphs, `/hotkeys`, and Alt+L; check terminal support.
Follow Omarchy does not change colors	Check that an active readable Omarchy palette exists; the configured fallback theme can still apply.

Logs, support, and cleanup

Default application logs are under `~/.pi-forge/logs`; paths can change under XDG/profile overrides. Saved sessions normally live under the active agent directory's `sessions` folder. Include the version, model/provider, task, error, and reproduction steps when reporting a problem. Do not include credentials from diagnostic dumps.

`pi-forge grievances` inspects recorded tool issues; pushing them is an external action. `pi-forge gc` cleans eligible stored data. Inspect their help before deleting or publishing anything. Statistics and session records can contain project text; preserve useful evidence before cleanup.

For offline data collection, use Real-work observation. It explains how to isolate a task's saved logs, summarize usage without another model request, and record whether the output actually met the goal.

12. Command reference and glossary

Terminal command reference

These commands run in your terminal. Use `pi-forge <command> --help` for options and exact argument formats. Utilities marked **Advanced** are primarily for administration, diagnostics, or developer workflows; some make model/service calls when executed. `launch` is the default command even though it is not listed as a separate public help entry.

Terminal command	Purpose
`pi-forge launch`	Start an interactive session, or use launch flags for a one-shot/protocol run.
`pi-forge acp`	Connect Pi Forge to a compatible editor using Agent Client Protocol.
`pi-forge auth-broker`	Advanced: manage a credential vault shared through a broker.
`pi-forge auth-gateway`	Advanced: provide an authenticated proxy or a scripted gateway for model requests.
`pi-forge agents`	Manage bundled task-agent definitions and inspect their configuration.
`pi-forge bench`	Advanced: measure model response delay, generation speed, caching, and parallel load.
`pi-forge browser-relay`	Start the local connection used to control existing Chrome tabs.
`pi-forge cleanse`	Find and repair project code diagnostics through coordinated workers.
`pi-forge collab`	Inspect local collaboration hosts and retrieve control or view-only links.
`pi-forge commit`	Generate/split commits and handle changelog updates according to the command options.
`pi-forge completions`	Print Bash, Zsh, or Fish shell completion setup.
`pi-forge compress`	Advanced: rewrite text into a denser instruction format and report omissions.
`pi-forge config`	Read/change settings, inspect their schema, or import OMP data.
`pi-forge dry-balance`	Advanced: preview account-balancing behavior without actual task work.
`pi-forge find`	Search code by a description of behavior, using a judge model.
`pi-forge gc`	Clean eligible stored application data.
`pi-forge grep`	Advanced: run the text-search tool directly for inspection or diagnosis.
`pi-forge gallery`	Advanced: preview interface components using deterministic examples.
`pi-forge git`	Open the Git interface to inspect differences, stage changes, and compose commits.
`pi-forge grievances`	Inspect, clean, or submit recorded tool issues.
`pi-forge images`	Inspect and diagnose image-publication backends and their stored data.
`pi-forge if-bench`	Advanced: test instruction following and working memory with a benchmark workload.
`pi-forge install`	Install/link an extension package; shortcut for plugin install/link.

Terminal command	Purpose
`pi-forge join`	Join a live collaborative session.
`pi-forge login`	Sign in to a supported model provider.
`pi-forge models`	List, search, and refresh available models.
`pi-forge plugin`	Install, link, remove, and manage plugin packages.
`pi-forge predict`	Advanced: compare input word-completion engines interactively.
`pi-forge ps`	Inspect/control supervised background services, including logs and restart.
`pi-forge say`	Speak supplied text through the local speech engine.
`pi-forge clip`	Upload a recording as a public clip to the inherited live service.
`pi-forge play`	Replay a screen recording; Space pauses and q quits.
`pi-forge share`	Share a saved session through an encrypted link.
`pi-forge setup`	Run onboarding or prepare dependencies for optional features.
`pi-forge shell`	Open an interactive shell console.
`pi-forge read`	Inspect what the file/URL read tool returns.
`pi-forge render`	Advanced: render a saved conversation and measure interface repaint timing.
`pi-forge skill`	Find, install, update, manage, or publish skills through the skill registry.
`pi-forge ssh`	Manage remote SSH host configurations.
`pi-forge stats`	Inspect local usage statistics or open the dashboard.
`pi-forge stream`	Broadcast rendered session screens and chat to the supported live service.
`pi-forge update`	Check for and install Pi Forge binary updates.
`pi-forge usage`	Show provider-reported limits for authenticated accounts.
`pi-forge tiny-models`	Download small local helper models used for titles, memory, or completion.
`pi-forge token`	Print a provider API key/OAuth token; its output is a credential.
`pi-forge toks`	Count file/text tokens using embedded offline tokenizers.
`pi-forge ttsr`	Inspect and test output-interruption rules.
`pi-forge worktree`	Create, list, or clear repository working copies.
`pi-forge search`	Advanced: exercise configured web-search providers directly.

Common launch options

Launch flags belong after `pi-forge`, or after its default `launch` command. This is a practical selection; `pi-forge --help` and CLI reference contain the complete launch surface.

Option	Purpose
`--cwd <dir>` / `--add-dir <dir>`	Choose the main/additional working directories.
`--model <id>`	Choose the active model.

Option	Purpose
`--smol`, `--slow`, `--plan`	Assign common model roles for this launch.
`--thinking <level>`	Choose reasoning effort.
`--profile <name>`	Select a named application profile.
`--config <file>`	Apply an extra settings overlay for this run.
`--continue` / `--resume [id]`	Reopen saved work.
`--fork <session>`	Create a separate conversation from saved work.
`--session-dir <dir>`	Use a specific session storage directory.
`--no-session`	Do not persist the conversation.
`--from-claude` / `--from-codex`	Import a supported foreign session.
`-p` / `--print`	Run a task without the interactive interface, then exit.
`--mode rpc`	Run the application through its scripted protocol.
`--tools <names>` / `--no-tools`	Limit or disable built-in tools.
`--no-lsp` / `--no-pty`	Disable language-service or interactive shell support.
`--approval-mode <mode>`	Select normal tool approval behavior.
`--auto-approve` / `--yolo`	Force automatic tier approval, subject to explicit policies.
`--advisor`	Enable an advisor for this launch.
`--prewalk` / `--prewalk-into <model-or-role>`	Arm a planning-to-execution model handoff.
`--plan-yolo`	Plan and then execute with automatic plan approval.
`--goal <objective>`	Start a new interactive goal session.
`--max-time <duration>`	Stop the process after a specified duration.
`--extension <path>` / `--trusted-extension <path>`	Load an extension or use a restricted explicit extension set.

Inside-Pi-Forge command reference

Type these in the conversation input, not the operating-system terminal. Type `/` to browse available commands and completions. Plugin/custom commands can add to this list. Some built-in commands require the interactive interface, a mode, or a configured integration.

Slash command	Purpose
`/security`	Plan, run, inspect, cancel, import/export, compare, or manage security scans.
`/settings`	Open the settings menu.
`/setup` (also `/providers`)	Open provider setup.
`/plan`	Toggle planning before execution.
`/plan-review`	Reopen the latest plan review.

Slash command	Purpose
`/vibe`	Toggle director mode with persistent worker sessions.
`/goal`	Manage a persistent objective: set, show, pause, resume, drop, or budget.
`/guided-goal`	Describe an objective through an interview before creating the goal.
`/loop`	Repeat the next prompt with optional count, duration, or command conditions.
`/queue`	Queue a follow-up for after the agent yields.
`/model` (also `/models`)	Select a model or change a configured model role.
`/switch`	Temporarily switch the session model.
`/fast`	Request supported priority/fast service; on, ultra, off, or status.
`/slow`	Request supported flex/low-priority service; on, off, or status.
`/skillful`	Toggle skill listings in the session instructions.
`/extended-context`	Toggle a supported larger context window.
`/computer`	Enable, disable, or inspect desktop automation for this session.
`/ratchet`	Build an evaluation and iterate on an AI workflow.
`/prewalk`	Arm a model handoff, or restart the planning/execution cycle.
`/modelpreset`	List, save, switch, or delete a model/effort preset.
`/effort`	Choose reasoning effort; available levels depend on the model.
`/prompt`	Forge: setup profiles; inspect/stats/compare; route/unroute; inspect Governor decisions.
`/advisor`	Configure, enable, disable, inspect, or dump advisor state.
`/export`	Export the session as HTML.
`/trace`	Open the session trace in the statistics dashboard.
`/dump`	Copy the transcript and write request diagnostics; all includes agent dumps.
`/share`	Create an encrypted session-sharing link.
`/collab`	Host a live session; view, list, status, or stop controls.
`/join`	Join live collaboration.
`/leave`	Leave a joined collaborative session.
`/browser`	Choose visible or headless browser mode.
`/copy`	Pick conversation text or code to copy.
`/open`	Open the last conversation link in a browser.
`/todo`	Inspect/edit the task list, or import/export and change selected items.
`/session`	Inspect session identity, delete a session, or manage its pin.
`/jobs`	Inspect asynchronous jobs; full shows more detail.
`/usage`	Inspect provider usage; reset adjusts supported local accounting.

Slash command	Purpose
`/stats`	Open local statistics.
`/changelog`	Read inherited upstream release history with the Forge version identified.
`/hotkeys`	Show effective keyboard shortcuts.
`/tools`	Show tools visible to the agent.
`/context`	Show estimated current context use.
`/extensions` (also `/status`)	Open the Extension Control Center.
`/agents`	Open the agent configuration hub.
`/git`	Open the Git interface.
`/hub`	Open the live Agent Hub.
`/branch` (also `/rewind`)	Return to an earlier message while retaining the old conversation branch.
`/fork`	Create a separate session from an earlier conversation point.
`/tree`	Navigate conversation branches.
`/login`	Sign in through a supported provider login.
`/logout`	Sign out of a provider.
`/mcp`	Add, test, authenticate, enable/disable, reconnect, or inspect external tool servers.
`/ssh`	Add, list, or remove SSH hosts.
`/new`	Start a new session.
`/fresh`	Reset provider stream state without clearing conversation history.
`/clear`	Clear active conversation context, keeping the session identity.
`/delete`	Delete the current saved session and start a new one.
`/compact`	Condense active context; alternate modes depend on provider/configuration.
`/shake`	Drop selected heavy context such as tool output, images, or thinking.
`/handoff`	Write a continuation summary and compact the session.
`/resume`	Open another saved session.
`/pin`	Pin/unpin the session in the recent list.
`/btw`	Ask a side question or inspect earlier side questions.
`/tan`	Run tangential work through a background agent.
`/omfg`	Turn a complaint about repeated behavior into an output-interruption rule.
`/cleanse`	Find and repair project diagnostics with workers.
`/retry`	Retry the last failed turn.
`/debug`	Open debugger/tool choices.
`/memory`	Inspect memory and run backend-specific maintenance or mental-model operations.

Slash command	Purpose
`/rename`	Rename this session, or request an automatically generated title.
`/move`	Move the session to a different directory.
`/wt` (also `/worktree`)	Move this session and its changes into a new worktree.
`/add-dir`	Add a workspace directory.
`/remove-dir`	Remove an additional workspace directory.
`/dirs`	List workspace directories.
`/exit`	Leave the application.
`/restart`	Restart Pi Forge with the same flags and resume this session.
`/marketplace`	Manage plugin sources, discovery, installation, and upgrades.
`/plugins` (also `/plugin`)	List, enable, or disable installed plugins.
`/reload-plugins`	Reload skills, commands, hooks, tools, agents, and MCP contributions.
`/skills`	Search, install, inspect installed, or update registry skills.
`/force` (also `/force:`)	Require a particular tool on the next turn, where supported.
`/live`	Start supported Codex-backed real-time voice.
`/record`	Start/stop local screen recording.
`/pause`	Freeze main/worker/advisor work until resumed.
`/quit` (also `/q`)	Leave the application.
`/review`	Bundled: review a selected change set and report prioritized findings.
`/annotate`	Bundled: attach notes to a diff, reply, message, file, or supplied text.
`/green`	Bundled: prepare a request to work through branch CI failures.

Tool and helper reference

Tools are capabilities the agent calls while doing your task. A tool may be essential, optional, discoverable behind `xd://`, or supplied by a plugin. With discoverable tools, `read xd://` lists devices and `write xd://<tool>` can invoke eligible operations; you can simply ask the agent to inspect the available devices. Disabling a feature or missing its dependency prevents activation regardless of a prompt route.

Tool or helper	Purpose and availability
`read`	Read files, directories, URLs, documents, archives, databases, and internal resources.
`write`	Create/replace content and access supported writable devices/resources.
`edit`	Apply line/content-anchored changes to files.
`grep`	Find text patterns across files or supported resources; normally enabled.
`glob`	Find paths by file-name patterns; normally enabled.
`find`	Find code by behavior, with model-assisted judgments; automatic availability.

Tool or helper	Purpose and availability
`ast_grep`	Search code structure; off by default.
`ast_edit`	Propose structural code rewrites; normally enabled/discoverable.
`bash`	Run shell commands, interactive tasks, and eligible background work.
`eval`	Run persistent Python/JavaScript code and configured helpers.
`lsp`	Navigate code and inspect language-service errors; needs a language server.
`debug`	Control a compatible debugger; needs a debug adapter/configuration.
`ida`	Inspect/script IDA databases; requires an IDA Pro/idalib installation.
`task`	Start and coordinate worker agents.
`wait`	Wait for jobs/results and handle peer communication.
`todo`	Track task phases and item status.
`ask`	Request structured user input when the interface permits it.
`web_search`	Search configured web providers; accounts/keys vary.
`github`	Perform GitHub operations; off by default, needs authentication.
`security_scan`	Plan/run supported security scans; off by default.
`checkpoint` / `rewind`	Save/restore conversation checkpoints; off by default.
`context_notes` / `new_context`	Forge experimental notebook and fresh-context request; off by default.
`retain` / `recall` / `reflect` / `memory_edit`	Store, find, reason over, and edit memory; backend-dependent.
`learn`	Record durable lessons; requires auto-learning/compatible memory behavior.
`manage_skill`	Create/update/delete managed skills under the eligible learning workflow.
`browser`	Eval helper for real browser tabs; not a standalone built-in AgentTool.
`computer`	Eval helper for desktop windows and input; off by default.
`generate_image`	Optional registered image tool/device; off by default, needs a supported provider.
`tts`	Optional registered speech-file tool/device; off by default.
`agent()` / `workpool()`	Eval helpers for reusable workers and dependency-aware concurrent jobs.
`judge()`	Eval helper to classify/rank items using a configured judge model.
Archive, document, Git, process, and ratchet helpers	Eval extensions for specific tasks; availability follows enabled helpers and dependencies.
Goal/Vibe control tools	Installed internally by the relevant mode; use the user-facing mode controls.
MCP and extension tools	Added by configured servers or installed packages; names and requirements vary.

Internal resources

An internal resource URL lets the agent inspect structured app data using familiar file tools. These are not public websites. Examples are `agent://` for worker results, `proc://` for supervised processes, `pr://` and `issue://` for GitHub resources, `conflict://` for merge conflicts, `skill://` for installed skills, `memory://` for supported memory, `history://` for recoverable conversation history, `cfg://` for settings, and `xd://` for discoverable devices. `ssh://` addresses configured remote hosts; optional `vault://` addresses Obsidian content.

Other providers/extensions can add schemes. The inherited `omp://` documentation namespace remains intentional. A resource's existence does not grant permission to change its target; some schemes are read-only or have specialized write rules.

Advanced embedding and administration

SDK: application developers can embed the agent in their own program and handle its events, tools, and UI. This is a programming interface, not another user mode. See SDK.

RPC: another process can control Pi Forge through structured messages over standard input/output. Launch with `--mode rpc`; keep protocol output separate from ordinary terminal text. See RPC.

ACP: a compatible editor can run the same coding agent as an Agent Client Protocol server. Use `pi-forge acp` and configure the editor according to its integration support. See CLI reference.

Authentication broker/gateway: administrators can centralize credentials or proxy requests without giving each client the underlying provider key. This needs deliberate server/network configuration; normal local use does not require it. See Broker and gateway.

Developer diagnostics: `bench`, `if-bench`, `gallery`, `render`, `predict`, `grep`, `read`, `search`, `dry-balance`, `toks`, and the debug tools examine performance, input/output, routing, rendering, or behavior. Benchmarks can make real model calls; offline token counting and existing-session analysis need not.

Advanced runtime options: speculative local reads (off by default), shell-command interception, context promotion to a larger model, alternative compaction backends, model retry/fallback chains, provider transport/privacy/sampling settings, per-agent model/prewalk/advisor overrides, output artifacts, image publication, and native terminal rendering are available through their settings and references. These tune particular bottlenecks; they are not required to start coding. `pi-forge config list` exposes the current settings catalog rather than freezing every advanced key in this guide.

Glossary

Term	In plain language
Agent	The AI model plus the tools/workflow that let it act on your project.
Provider	The service or local server running a model.
API key / OAuth	A service credential / a supported account sign-in method.
Token	A small unit of text a model processes.
Context / context window	Material seen by the next model request / its size limit.
Compaction	Condensing active conversation context to make room.
Session / branch / fork	Saved conversation / an alternate path / a separate conversation derived from it.
Model role / preset	A model assignment for a job / a saved set of model and effort choices.
Prompt profile / module	A choice of instruction-loading policies / one instruction section.
Tool / capability	An operation the agent can call / a group of relevant guidance or functionality.
Skill / plugin / marketplace	A playbook / a bundle of capabilities / a place to discover bundles.
MCP	A standard way to connect external tool servers.
LSP / debug adapter	Editor-style code intelligence / a connection to a debugger.
Subagent / work pool	Another agent handling part of a task / a shared queue of worker jobs.
Worktree / isolation	Another repository working copy / keeping worker edits apart.
Integration gate	Checks on a worker batch before its changes are accepted.
Governor / verification band	Adaptive task policy / a level of checking effort.
Artifact	Saved output too large or specialized to show fully in the conversation.
TTS / STT	Turning text into speech / speech into text.
Headless / CDP	Running without a visible interface / a browser-control connection.
SDK / RPC / ACP	Embed in a program / control through messages / connect to an editor.

Further reading and guide provenance

This guide covers the Pi Forge 0.1.1 implementation, including the OMP 18.6.2 integration. It was checked against the public CLI command table, built-in slash-command registry, bundled commands, tool factories and optional helper registrations, settings definitions, README, and domain documentation. Hidden worker entry points and package-internal APIs are excluded; the public modes they support are included. Third-party plugins can add features beyond this inventory.

Some detailed inherited documents still use the word OMP, `omp` commands, or `.omp` paths. For ordinary Pi Forge use, substitute `pi-forge` and the active Forge configuration paths. Keep intentional external service names, legacy compatibility sources, and protocol/resource identifiers as documented.

Useful next references: CLI flags, Settings, Models, Providers, Keybindings, Sessions, Compaction, Memory, Browser, Desktop automation, Skills, MCP, Extensions, Themes, and Upstream reconciliation.